# Cours : Le principe de substitution de Liskov (LSP)

## 1. Contexte dans SOLID

Le **LSP** est le troisième principe de **SOLID**. Il garantit la cohérence des hiérarchies d'héritage : étendre un type ne doit pas casser le comportement attendu du système.

## 2. Définition

"Les objets d'un programme doivent pouvoir être remplacés par des instances de leurs sous-types sans altérer la validité du programme." — Barbara Liskov (1987).

## 3. Objectif

Préserver le contrat du type parent : le code qui attend le parent fonctionne de la même manière avec tout sous-type. C'est la base d'un polymorphisme fiable.

## 4. Violations typiques

• Restreindre un comportement promis par le parent (préconditions plus fortes, postconditions affaiblies).
• Lancer des exceptions inattendues.
• Modifier des invariants du parent.

## 5. Cas Rectangle/Carré : bonne pratique

Le carré ne respecte pas le contrat implicite d'un rectangle (largeur/hauteur indépendantes) : substituer **Square** à **Rectangle** casse le code client ⇒ violation du LSP.

■ Recommandation : éviter l'héritage *Square extends Rectangle*. Utiliser une **interface** (ex. **Shape** avec *area()*, *perimeter()*) et conserver **deux classes distinctes** (*Rectangle*, *Square*) qui implémentent le même contrat. On garde les invariants propres à chaque forme et on respecte le LSP.

## 6. Appliquer le LSP au quotidien

• Préférer composition/abstraction à un héritage douteux.
• Clarifier le contrat (interfaces, classes abstraites) et le tester.
• Réviser une hiérarchie si un sous-type doit changer trop de règles du parent.

## 7. Conclusion

Les sous-types doivent être substituables aux super-types. Le LSP garantit un polymorphisme sûr et prévisible et s'aligne naturellement avec SRP et OCP.